

1 Problembeskrivelse

Jyllands-Posten anvender i stigende grad kampagnebaserede abonnementer til at tiltrække nye digitale kunder. Selvom disse kampagner genererer en høj tilgang af abonnenter, oplever virksomheden en markant churn blandt kampagneabonnenter kort tid efter kampagneperiodens udløb.

Den høje churn-rate udfordrer abonnementsforretningens langsigtede bæredygtighed og vanskeliggør en effektiv prioritering af fastholdelses- og genkøbsindsatser. Jyllands-Posten råder over omfattende data om abonnenternes adfærd, historik og engagement i kampagneperioden, men disse data anvendes i begrænset omfang til systematisk og forklarlig forudsigelse af churn.

Der er derfor behov for at undersøge, i hvilken grad Machine Learning-modeller kan anvendes til at forudsige churn blandt kampagneabonnenter baseret på deres adfærd og historik, samt hvordan disse prædiktionsmodeller kan gøres forståelige og handlingsorienterede for forretningen.

For at understøtte Jyllands-Posten anvendelse skal modelresultater og centrale churn-drivere visualiseres i en intuitiv dashboard-løsning, der kan bruges af ikke-tekniske interessenter til at identificere risikokunder, vigtige mønstre og relevante indsatsområder. Både i relation til fastholdelse og planlægning af fremtidige kampagner.

1.1 Afgrænsning

I projektet fokuseres der udelukkende på abonnenternes adfærd og historik i selve kampagneperioden. Analysen baseres på data, der er tilgængelige fra købstidspunktet og frem til kampagneperiodens udløb.

Projektet afgrænses fra at analysere kundernes adfærd efter overgang til et ordinært abonnement samt fra at inddrage længerevarende churn-forløb efter kampagneperiodens afslutning.

1.2 Datakilder

Projektet baseres på de udleveret datafiler fra Jyllands Posten, som indeholder tre datasæt:

behavior.csv: kundeadfærd i kampagneperioden

Cancellation.csv: churn-label + begrundelser

Subscription.csv: abonnements- og kampagneinfo

2 Datajura

Når Jyllands-Posten arbejder med abonnentdata og digitale brugerdata, er datajura en vigtig del af projektet. Det at sikre data bliver brugt lovligt og ansvarligt. De nye oplysninger fra JP/Politikens Hus viser, at data kan indgå i flere sammenhænge, fx abonnement, brugeradfærd, annoncering, kampagnestyling og målretning. Derfor er det vigtigt at have styr på behandlingsgrundlag, samtykke, fælles dataansvar og sikkerhed (JP/Politikens Hus, 2025).

3 GDPR og etik

GDPR er en af de vigtigste ting at have styr på, når man arbejder med persondata. I denne case kan der være tale om abonnementsdata, adfærdsdata, kampagnedata og cookie-baserede ID'er. Jyllands-Posten skal derfor have et lovligt grundlag for at behandle data. Det kan fx være samtykke eller legitim interesse jf. artikel 6 i GDPR (JP/Politikens Hus, 2025).

I JP/Politikens Hus' retningslinjer står der, at de bruger en Consent Management Platform, også kaldet CMP, til at indhente samtykke fra brugerne. Det betyder, at brugeren skal kunne tage stilling til, hvilke former for databehandling de accepterer. Samtykke er især vigtigt, hvis data bruges til profilering, målretning eller

personaliserede annoncer. Hvis der bruges legitim interesse, skal det kunne begrundes, og der skal laves en vurdering af virksomhedens interesse over for brugerens rettigheder (JP/Politikens Hus, 2025).

Derudover gælder princippet om dataminimering jf. artikel 5 i GDPR. Det betyder, at Jyllands-Posten kun må bruge de data, der er nødvendige for formålet. Hvis formålet er at analysere churn, skal man derfor overveje, hvilke variable der faktisk er relevante. Man må ikke bare tage alle data med, fordi de findes. Samtidig gælder formålsbegrænsning jf. artikel 5, som betyder, at data kun må bruges til det formål, de er indsamlet til. Hvis data fx er indsamlet til abonnementshåndtering, kan de ikke uden videre bruges til avanceret profilering uden et gyldigt grundlag (JP/Politikens Hus, 2025).

Kunderne har også rettigheder efter GDPR jf. artikel 13-22. De har fx ret til indsigt, sletning, berigtigelse og i nogle tilfælde dataportabilitet. Det betyder, at Jyllands-Posten skal have systemer og processer, der kan håndtere, hvis en kunde ønsker at se eller få slettet sine data (JP/Politikens Hus, 2025).

3.1 Fælles dataansvar

En vigtig ny oplysning er, at JP/Politikens Hus i nogle situationer arbejder med fælles dataansvar jf. artikel 26 i GDPR. Det betyder, at JPPOL og en annoncør eller et medie bureau sammen kan være ansvarlige for bestemte dele af databehandlingen. Det sker fx, når en annoncør køber annoncering på JPPOL-sites og bruger egne eller tredjeparts teknologier til at indsamle data om brugere (JP/Politikens Hus, 2025).

Her er ansvarsfordelingen vigtig. JPPOL er ansvarlig for, at der findes et gyldigt behandlingsgrundlag, og at brugerne får information om behandlingen. Indkøberen eller annoncøren er ansvarlig for den videre behandling af data, efter at data er blevet sendt videre. Det betyder, at begge parter har ansvar, men ikke nødvendigvis for det samme (JP/Politikens Hus, 2025).

Hvis en bruger henvender sig om sine rettigheder, skal parterne kunne sende henvendelsen videre til den rigtige part hurtigst muligt og senest inden for 72 timer. Det viser, at dataansvar ikke kun handler om regler, men også om klare processer i praksis (JP/Politikens Hus, 2025).

3.2 AI etik og lovgivning

Når Jyllands-Posten bruger AI og Machine Learning til at forudsige churn, opstår der også etiske overvejelser. En churn-model kan hjælpe virksomheden med at finde kunder, der er i risiko for at opsiges deres abonnement. Det kan være positivt, fordi virksomheden kan give mere relevante tilbud og bedre kommunikation. Men det kan også blive problematisk, hvis modellen bruges til at presse eller manipulere kunder.

En vigtig risiko er bias. Hvis modellen trænes på historiske data, kan den komme til at videreføre gamle skævheder. Hvis unge kunder fx tidligere har haft høj churn, kan modellen komme til at vurdere unge som mere risikable, selvom årsagen måske ikke er fair. Derfor er det vigtigt at teste modellen og kunne forklare de vigtigste faktorer bag en churn-vurdering.

Transparens er også vigtig. Hvis kunder bliver behandlet forskelligt på baggrund af en model, bør virksomheden kunne forklare hvorfor. Det betyder ikke, at kunden skal se hele modellen, men der skal være en forståelig forklaring på, hvorfor modellen vurderer en kunde som høj risiko.

I forhold til AI-lovgivning, fx EU AI Act, vil en churn-model typisk ikke være højrisiko. Men Jyllands-Posten bør stadig dokumentere, hvordan modellen er udviklet, hvilke data der er brugt, hvordan den er evalueret, og hvad formålet er. Hvis modellen senere bruges til automatiske beslutninger, fx ændring af priser eller vilkår, kan risikoen blive højere, og så vil kravene også blive større.

3.3 Cybersikkerhed

Cybersikkerhed er også en central del af datajuraen. Når Jyllands-Posten arbejder med mange abonnent- og brugerdata, skal de beskytte data mod datalæk og uautoriseret adgang. I retningslinjerne fremgår det, at der skal bruges passende tekniske og organisatoriske foranstaltninger jf. artikel 32 i GDPR. Det gør de ved pseudonymisering, adgangsstyring og løbende test af sikkerheden (JP/Politikens Hus, 2025).

Det handler også om at sikre fortrolighed. Data skal være beskyttet, de må ikke ændres forkert, og systemerne skal kunne fungere stabilt. Hvis der sker et databrud, skal den part, hvis sikkerhed er berørt, anmelde bruddet til Datatilsynet jf. artikel 33, og i nogle tilfælde informere brugerne jf. artikel 34 (JP/Politikens Hus, 2025).

3.4 Databehandlere og tredjepartsteknologier

De nye oplysninger viser også, at JPPOL og samarbejdspartnere kan bruge databehandlere og tredjepartsteknologier. Det kan fx være annonceplatforme, analyseværktøjer eller andre teknologipartnere. Her er det vigtigt, at der findes gyldige databehandleraftaler jf. artikel 28 i GDPR, og at leverandørerne kan leve op til reglerne (JP/Politikens Hus, 2025).

Derudover fremgår det, at JPPOL-data ikke må kopieres til egne systemer eller bruges på andre sites end JPPOL-sites. Det er vigtigt, fordi det beskytter JPPOL's førstepartsdata og mindsker risikoen for, at data bliver brugt uden for det oprindelige formål (JP/Politikens Hus, 2025).

4 Databehandling TODO

Der starts med at rense og klargøres data, så vi har et Datasæt for Jyllands Postens abonnementsdata der er klar til udviklingen af en Machine Learning model.

Først indlæser vi de nødvendige pakker og de udleveret datasæt, hvor vi arbejder med både abonnementsdata, afmeldingsdata og affærsdata. Herefter sørger vi for, at hver kunde kun optræder én gang ved at fjerne dubletter på `pseudo_id`. Det gør vi for at undgå fejl i analysen og sikre, at hver observation repræsenterer én kunde.

Derefter fletter vi datasættene sammen, så vi får alle relevante informationer samlet ét sted. Samtidig omdanner vi datoer til rigtige datoformater, så de kan bruges korrekt i beregninger. Det er vigtigt, fordi datoerne bruges til at definere churn.

Herefter laver vi de vigtigste variable. Først definerer vi `churn`, som viser, om kunden ikke fortsætter efter kampagnen. Hvis abonnementet stopper før eller ved kampagnens slutdato, er det `churn`. Hvis det fortsætter, er det ikke `churn`. Derefter laver vi `continued_subscription`, som bare er det modsatte af `churn`. Til sidst laver vi `early_churn`, som viser de kunder, der godt nok fortsætter, men opsiger kort tid efter. Her har vi sat grænsen til 90 dage efter kampagnen.

Efter det går vi i gang med datarensning og feature engineering. Vi fjerner `pseudo_id`, fordi det kun er en identifikator. Vi omdanner datoer og fjerner rækker med manglende værdier, så vi arbejder med valide data. Samtidig gør vi kategoriske variable som fx køn og afmeldingsårsag klar som faktorer. Manglende værdier bliver erstattet med ikke oplyst, så data stadig giver mening.

Derudover laver vi nye variable som alder og kundetid, som kan være vigtige for at forklare churn. Kundetid bliver også opdelt i grupper, så det er lettere at analysere. Til sidst fjerner vi de variable, vi ikke længere har brug for.

Til sidst tjekker vi datasættet og ser på fordelingen af churn og de andre variable. Til sidst gemmer vi det rensede datasæt i en RDS fil, så det er klar til videre analyse og modellering.

4.1 Machine learning modellen

I denne del af projektet har vi udviklet to Machine Learning-modeller, som tager udgangspunkt i Jyllands-Postens forretningsproblem omkring churn. Den første model har til formål at forudsige, hvilke kunder der cherner direkte efter kampagneperioden, og dermed også hvilke kunder der fortsætter. Den anden model bygger videre på den første og fokuserer kun på de kunder, der faktisk fortsatte efter kampagnen, for derefter at forudsige hvilke af dem der alligevel cherner efter kort tid. På den måde arbejder vi i to trin, som passer direkte til problemformuleringen i casen.

Vi starter med at indlæse det rensede datasæt `model_data.rds`, som allerede er klargjort gennem data cleaning og feature engineering. Derefter laver vi et nyt datasæt til model 1, hvor vi bevidst fjerner variable, som kan give leakage. Det gælder blandt andet `subscription_cancel_date`, `type`, `reason`, `expiration_date`, `first_campaign_day`, `last_campaign_day` og `continued_subscription`. Grunden til, at de bliver fjernet, er, at de enten direkte afslører udfaldet eller ligger for tæt op ad target-variablen. Hvis vi beholdt dem, ville modellen få information, den i praksis ikke burde have adgang til på prediction-tidspunktet, og så ville performance blive kunstigt høj. Vi beholder heller ikke `pseudo_id`, da det kun er en teknisk identifikator og ikke en reel forklarende variabel.

Selve target-variablen i model 1 er `churn`, som bliver lavet til en faktor med niveauerne 0 og 1. Herefter splitter vi datasættet i træningsdata og testdata med en 80/20 fordeling, hvor vi bruger stratificering på `churn`. Det gør vi for at sikre, at fordelingen mellem `churn` og ikke-`churn` ligner hinanden i både `train` og `test`. Efter det laver vi 5-fold cross validation på træningsdata, igen med stratificering. Formålet med det er at få en mere stabil og troværdig vurdering af modellernes performance under tuning.

I recipe-delen for model 1 forbereder vi data systematisk. Først fjerner vi `early_churn`, fordi den variabel er direkte relateret til fremtidig adfærd og derfor vil skabe leakage i `churn`-modellen. Derefter bruger vi `step_novel`, så modellen kan håndtere nye kategorier i testdata. Vi laver dummy-encoding af alle kategoriske variable med one-hot encoding, så modellerne kan arbejde numerisk med dem. Herefter fjerner vi zero variance variable, altså variable uden reel variation, og normaliserer de numeriske variable. Til sidst bruger vi `step_downsample(churn)`, fordi `churn`-klasserne sandsynligvis er ubalancerede. Downsampling gør, at majoritetsklassen reduceres i træningsdata, så modellerne ikke bare lærer at gætte den største klasse hele tiden.

Vi tester fem forskellige klassifikationsmodeller. Den første er et decision tree, hvor vi tuner `tree_depth`, `min_n` og `cost_complexity`. Den anden er logistisk regression med regularisering via `glmnet`, hvor vi tuner `penalty` og `mixture`. Den tredje er random forest med `ranger`, hvor vi tuner `mtry` og `min_n`, mens antal træer er sat til 500. Den fjerde er en SVM med radial basis function, hvor vi tuner `cost` og `rbf_sigma`. Den femte er XGBoost, hvor vi tuner blandt andet antal træer, trædybde, learning rate og `mtry`. På den måde sammenligner vi både simple, fortolkelige og mere avancerede modeller.

Alle modeller bliver samlet i et `workflow_set`, så de kan køres igennem den samme preprocessing pipeline. Derefter tuner vi dem med `tune_grid` på vores cross validation folds. Vi bruger flere evalueringsmål: `roc_auc`, `accuracy`, `f_meas`, `sens` og `spec`. Det er vigtigt, fordi `accuracy` alene ofte er misvisende ved ubalancerede klasser. Vi vælger den bedste model ud fra `f_meas`, da den balancerer `precision` og `recall` og derfor er særlig relevant i en churn-case, hvor det er vigtigt både at finde churn-kunder og undgå for mange falske alarmer.

Når den bedste model er fundet, henter vi de bedste hyperparametre og finaliserer workflowet. Derefter laver vi `last_fit` på det oprindelige split, så modellen bliver trænet på hele træningsdatasættet og evalueret på testdatasættet. Til sidst indsamler vi `metrics`, laver en confusion matrix og en ROC-curve. Confusion matrixen giver os et konkret billede af, hvor mange kunder modellen klassificerer korrekt og forkert, mens ROC-kurven viser modellens evne til at skelne mellem `churn` og ikke-`churn` på tværs af thresholds.

Model 2 er bygget op på næsten samme måde, men her fokuserer vi kun på kunder, som allerede har `continued` efter kampagnen, altså hvor `churn == 0`. Formålet med denne model er at identificere `early churn`, altså kunder som først fortsætter, men alligevel falder fra kort tid efter. Det gør modellen mere forretningsmæssigt relevant, fordi Jyllands-Posten så ikke kun kan forstå det første frafald, men også den gruppe af kunder, der ser ud til at være vundet, men som stadig er ustabile.

I model 2 laver vi derfor et nyt datasæt `early_data`, hvor target-variablen er `early_churn`. Vi splitter igen i `train` og `test` med stratificering og laver 5-fold cross validation. I recipe'en fjerner vi `churn`, fordi det er parent target og ikke skal bruges som predictor i `early churn`-modellen. Ellers følger preprocessing stort set samme logik som i model 1: håndtering af nye kategorier, dummy-encoding, fjernelse af zero variance og normalisering. Her har vi valgt ikke at bruge downsampling i den nuværende version, men det er kommenteret i koden, så det kan tilføjes senere, hvis klasseubalancen viser sig at være et større problem.

Vi genbruger de samme fem modeltyper, så vi kan sammenligne resultaterne på tværs af de to opgaver. Også her vælger vi den bedste model på baggrund af `f_meas`, finaliserer workflowet og evaluerer på testdatasættet. Resultatet bliver igen vurderet med `metrics` og confusion matrix.

Samlet set er styrken ved denne ML-opsætning, at den er systematisk, sammenlignende og forholdsvis robust. Vi arbejder ikke kun med én model, men tester flere forskellige algoritmer. Vi håndterer leakage bevidst, laver

preprocessing gennem recipes, bruger cross validation og hyperparameter tuning, og evaluerer modellerne med flere relevante metrics. Samtidig er modellen bygget direkte op omkring Jyllands-Postens forretningsproblem, fordi vi både forudsiger det første churn efter kampagnen og det efterfølgende early churn blandt de kunder, der i første omgang bliver fastholdt. Det gør løsningen mere brugbar i praksis, fordi resultaterne både kan bruges til at identificere risikokunder og til at planlægge mere målrettede retention-indsatser.

4.2 Kundeadfærdsanalyse

I dette afsnit kigger vi på kundeadfærd ud fra vores data i behavior.csv. Formålet er at forstå, hvordan brugerne opfører sig på siden, og om vi kan finde nogle mønstre, som kan bruges videre i forhold til churn.

Først laver vi data om til kundeniveau, så hver kunde kun har én række. Her laver vi nogle variable, som beskriver adfærden, fx hvor mange besøg en bruger har (n_visits), hvor langt de scroller (avg_scroll), og om de bruger mobil eller computer. Vi ser også på, hvor meget de er inde på paywall indhold, hvor de kommer fra (search eller intern), og hvor mange forskellige sider de besøger. Det giver et ret godt billede af, hvor aktive og engagerede kunderne er.

Derefter standardiserer vi data og laver en PCA. Det gør vi for at gøre data mere simpelt og samle informationen i færre variable, som stadig forklarer det meste. På den måde bliver det nemmere at arbejde videre med.

Efter det laver vi klyngeanalyse, hvor vi prøver at opdele kunderne i 3 grupper baseret på deres adfærd. Vi bruger både hierarkisk clustering og k-means for at være mere sikre på vores opdeling. Når vi har klyngerne, kigger vi på gennemsnit for hver gruppe, så vi kan forstå forskellene og lave en profil af hver kundetype.

Til sidst tester vi, om der faktisk er forskelle mellem klyngerne med ANOVA, og vi laver også nogle plots for at visualisere det.

Samlet set hjælper analysen os med at forstå kundeadfærd bedre og gør det muligt at opdele kunderne i forskellige typer, som vi senere kan bruge i vores churn analyse.

4.3 Evaluering af kode

I denne del af projektet evaluerer vi vores Machine Learning-modeller for at vurdere, hvor godt de faktisk kan forudsige churn. Vi bruger flere forskellige metrics som accuracy, ROC-AUC, sensitivity og F1-score. Grunden til, at vi ikke kun bruger én metric, er, at datasættet er skævt fordelt, hvor der typisk er flere kunder, der ikke cherner, end kunder der cherner. Derfor kan accuracy alene give et misvisende billede af modellens performance.

Vi har især fokus på F1-score, fordi den balancerer mellem precision og recall. I en forretningskontekst er det vigtigt både at identificere de kunder, der faktisk cherner (undgå false negatives), og samtidig undgå at ramme for mange kunder forkert med retention-indsatser (false positives). Hvis modellen fx overser churn-kunder, mister virksomheden potentielle indtægter, og hvis den rammer forkert, kan det føre til unødvendige marketingomkostninger.

Derudover bruger vi ROC-AUC til at vurdere modellens evne til at skelne mellem churn og ikke-churn på tværs af forskellige thresholds. Sensitivity er også relevant, da den viser, hvor stor en andel af churn-kunderne modellen faktisk fanger.

Resultaterne viser, at ensemble-modeller som random forest og XGBoost generelt performer bedst. Det giver god mening, da de er bedre til at håndtere komplekse sammenhænge og mange variable i data. Samlet set giver evalueringen et solidt grundlag for at vælge den mest brugbare model i forhold til Jyllands-Postens forretningsbehov.

5 Datavisualisering

I dette projekt spiller datavisualisering en central rolle, fordi det er med til at omsætte vores analyser og modeller til noget, Jyllands-Posten rent faktisk kan bruge i praksis. Når man arbejder med churn, kundeadfærd

og segmentering, kan resultaterne hurtigt blive tekniske og svære at læse direkte ud fra tabeller eller modeloutput. Derfor bruger vi både Shiny og Power BI til at visualisere data, så resultaterne bliver mere overskuelige, interaktive og lettere at formidle til virksomheden.

I vores kode arbejder vi med flere forskellige typer visualiseringer alt efter, hvad vi gerne vil undersøge. I adfærdsanalysen bruger vi fx histogrammer til at undersøge fordelingen af variable som antal besøg, scroll depth og antal unikke sider. Formålet med histogrammerne er at få et overblik over, hvordan data fordeler sig, og om der fx er skævheder, outliers eller meget store forskelle mellem kunderne. Det er vigtigt i en tidlig fase, fordi det giver en bedre forståelse af datasættet og hjælper os med at vurdere, hvordan data skal behandles videre.

Derudover bruger vi boxplots til at sammenligne forskelle mellem de kundeklynger, vi finder i vores segmentering. I koden laver vi fx boxplots for `n_visits`, `avg_scroll`, `share_mobile` og `share_search` fordelt på de klynger, der er fundet med k-means. De diagrammer er gode, fordi de ikke bare viser gennemsnit, men også spredning, median og eventuelle outliers. Det gør det lettere at se, om en bestemt klynge fx har højere aktivitet, større mobilandel eller mere trafik fra search end de andre grupper. På den måde bliver det nemmere at lave profiler af kundetyperne.

Vi bruger også PCA som en del af visualiseringen. Her laver vi først en standardisering af data og derefter en Principal Component Analysis, så vi kan reducere antallet af variable og samle informationen i færre komponenter. Til dette bruger vi blandt andet screeplots og biplots. Screeplottet hjælper os med at vurdere, hvor mange komponenter der giver mening at arbejde videre med, fordi det viser, hvor meget variation hver komponent forklarer. Biplottet gør det muligt at se relationen mellem observationer og variable i et reduceret rum, hvilket giver en visuel forståelse af, hvordan kundeadfærden hænger sammen. Det er især nyttigt før klyngeanalysen, fordi vi kan få en idé om, om der faktisk er naturlige grupper i data.

I forhold til selve segmenteringen bruger vi også visualisering i forbindelse med hierarkisk clustering. Her plotter vi dendrogrammet fra `hclust`, så vi kan se, hvordan kunderne grupperer sig, og hvor det giver mening at skære træet for at få et bestemt antal klynger. Det er med til at underbygge, hvorfor vi vælger fx 3 klynger i vores videre analyse.

Når vi går fra analyse til formidling, bruger vi Shiny til at gøre visualiseringerne interaktive. Det betyder, at brugeren selv kan vælge variable, filtre og visninger. Det gør løsningen mere fleksibel, fordi Jyllands-Posten ikke bare ser et statisk billede, men selv kan udforske data. I Shiny kan man fx forestille sig, at brugeren vælger en bestemt kundeklynge og derefter ser udviklingen i churn, fordeling af adfærd eller forskelle i abonnementsmønstre. Det giver meget mere værdi end en fast rapport, fordi virksomheden selv kan stille spørgsmål til data og få svar med det samme.

Power BI bruger vi mere til dashboards og ledelsesoverblik. Hvor Shiny er god til interaktiv analyse, er Power BI stærk til at samle KPI'er og vigtige resultater i et mere forretningsorienteret format. Her kan vi fx vise churn-rate, andel continued subscription, early churn, segmentfordelinger og udviklinger over tid i et dashboard. Vi kan også bruge slicers og filtre, så Jyllands-Posten kan se resultater for bestemte kundegrupper eller perioder. Diagrammer i Power BI kan fx være søjlediagrammer til at sammenligne churn mellem segmenter, linjediagrammer til udvikling over tid og kort eller tabeller til at vise mere detaljeret information.

Det vigtige er derfor ikke kun, at vi visualiserer data, men også hvordan vi gør det. Histogrammer hjælper os med at forstå fordelingen i data. Boxplots hjælper os med at sammenligne grupper. Screeplots, biplots og dendrogrammer hjælper os med at forstå vores PCA og klyngeanalyse. Shiny giver mulighed for interaktiv udforskning, mens Power BI giver et hurtigt og professionelt overblik over de vigtigste resultater.

Samlet set er datavisualisering en meget vigtig del af projektet, fordi det binder analyse og forretning sammen. Det gør vores resultater mere forståelige og mere anvendelige for Jyllands-Posten. Ved at kombinere kodebaserede visualiseringer i R med interaktive løsninger i Shiny og dashboards i Power BI får vi både en analytisk og en praktisk måde at arbejde med resultaterne på. Det gør, at vores projekt ikke kun viser, hvad modellen kan, men også hvordan indsigterne kan bruges i praksis.

5.1 Evaluerings af forretningsmæssige fordele

I en forretningsmæssig sammenhæng kan denne analyse direkte være med til at adressere Jyllands Postens problem med høj churn efter kampagneperioder. Ved at bruge Machine Learning til at forudsige churn, får

virksomheden mulighed for at identificere hvilke kunder, der er i risiko for at opsige deres abonnement allerede inden det sker. Det gør det muligt at arbejde proaktivt i stedet for reaktivt.

Samtidig giver analysen indsigt i, hvilke faktorer der driver churn. Ved at kombinere abonnementsdata og adfærdsdata kan Jyllands Posten forstå, hvilke typer kunder der cherner, og hvad der kendetegner dem. Fx kan det være lav aktivitet, få besøg eller manglende engagement med indhold. Denne viden gør det muligt at målrette indsatsen, så man ikke bruger ressourcer på alle kunder, men fokuserer på dem med størst risiko.

Kundesegmenteringen spiller også en vigtig rolle, da den opdeler kunderne i forskellige grupper baseret på deres adfærd. Det gør det muligt at tilpasse både kommunikation, tilbud og onboarding. Fx kan nye kunder med lav aktivitet få ekstra hjælp til at komme i gang, mens mere aktive kunder kan få mere relevant indhold eller premium tilbud. På den måde kan Jyllands-Posten skabe mere værdi for den enkelte kunde og dermed øge sandsynligheden for fastholdelse.

Derudover gør visualiseringerne i Shiny og Power BI det muligt at omsætte analysen til et konkret værktøj for forretningen. I stedet for kun at have tekniske modelresultater, får Jyllands Posten et dashboard, hvor de kan følge churn, identificere risikokunder og se mønstre i data. Et gør løsningen anvendelig for ikke-tekniske brugere og understøtter bedre beslutningstagning. Ud fra det kan det konkluderes, at Jyllands Posten kan arbejde mere datadrevet med både fastholdelse og fremtidige kampagner. I stedet for at fokusere på volumen af nye abonnenter, kan de i højere grad fokusere på kvaliteten af kunderne og deres langsigtede værdi. Dette kan være med til at reducere churn, øge kundeloyaliteten og skabe en mere bæredygtig abonnementsforretning.

6 Kode

6.1 Datarensning

```
pacman::p_load(tidyverse, lubridate, forcats, readr)

# 1. Indlæs data
cancellation <- read.csv("data/cancellation.csv")
subscription <- read.csv("data/subscription_v2.csv", sep = ";")

# Sæt import_date baseret på nyeste data i filen
import_date <- as.Date("2026-03-24")

# 2. Behold seneste observation pr. bruger
cancellation <- cancellation %>%
  mutate(expiration_date = ymd(expiration_date)) %>%
  group_by(pseudo_id) %>%
  slice_max(expiration_date, n = 1, with_ties = FALSE) %>%
  ungroup()

subscription <- subscription %>%
  mutate(subscription_cancel_date = dmy(subscription_cancel_date)) %>%
  group_by(pseudo_id) %>%
  slice_max(subscription_cancel_date, n = 1, with_ties = FALSE) %>%
  ungroup()

# 3. Merge + churn logik
merged_data <- subscription %>%
  left_join(cancellation, by = "pseudo_id") %>%
  mutate(
    subscription_cancel_date = as.Date(subscription_cancel_date),
    first_campaign_day = dmy(first_campaign_day),
```

```

    last_campaign_day = dmy(last_campaign_day),
    expiration_date = as.Date(expiration_date)
  ) %>%
  mutate(
    churn = case_when(
      is.na(expiration_date) ~ 0,
      expiration_date <= last_campaign_day ~ 1,
      expiration_date > last_campaign_day ~ 0
    )
  ) %>%
  mutate(
    continued_subscription = 1 - churn
  ) %>%
  mutate(
    early_churn = case_when(
      continued_subscription == 1 &
        !is.na(expiration_date) &
        expiration_date <= last_campaign_day + 90 ~ 1,
      continued_subscription == 1 ~ 0,
      continued_subscription == 0 ~ 0
    )
  )
)

# 4. Feature engineering + rensning + JUSTERING AF AKTIVE DAGE
model_data <- merged_data %>%
  mutate(
    # Parse datoer
    order_date = as.Date(ymd_hms(order_date)),
    birthdate = as.Date(parse_date_time(birthdate, orders = c("dmy", "ymd", "mdy"))),
    usr_created = dmy(usr_created),

    # Afledte variable
    age = as.numeric(import_date - birthdate) / 365,
    kundetid_dage = as.numeric(order_date - usr_created)
  ) %>%

  # Fjern ugyldige observationer
  filter(
    !is.na(birthdate),
    !is.na(usr_created),
    !is.na(order_date),
    previous_trials <= 50,
    age >= 15,
    age <= 105,
    kundetid_dage >= 0
  ) %>%

  # Rens kategoriske variable
  mutate(
    churn = factor(churn),
    type = fct_na_value_to_level(factor(type), "Ingen afmelding"),
    reason = fct_na_value_to_level(factor(reason), "Ingen afmelding"),
    koen = fct_recode(factor(koen), "Ikke oplyst" = ""),
    permission_given_order = factor(permission_given_order),
    permission_given_today = factor(permission_given_today),

```

```

    permission_given_order = factor(if_else(permission_given_order == "true", 1, 0)),
    permission_given_today = factor(if_else(permission_given_today == "true", 1, 0)),

    expiration_date = replace_na(expiration_date, as.Date("3000-01-01"))
  ) %>%

# Kundelevetid grupper
mutate(
  kundetid_gruppe = case_when(
    kundetid_dage <= 7 ~ "0-7 dage",
    kundetid_dage <= 30 ~ "8-30 dage",
    kundetid_dage <= 180 ~ "1-6 måneder",
    TRUE ~ "6+ måneder"
  ) %>% factor()
) %>%

# Fjern hjælpe-variable og rå datoer
select(
  -kundetid_dage,
  -order_date,
  -birthdate,
  -usr_created,
)

# 5. Tjek resultatet
glimpse(model_data)

```

```

Rows: 1,259
Columns: 22
$ pseudo_id          <chr> "003a92354fb88e0e3e411bc290291b6a", "004b2a16~
$ account_active_days <int> 387, 1330, 61, 92, 581, 1424, 1501, 184, 1256~
$ subscription_cancel_date <date> 2025-05-11, 2025-04-29, 2025-03-18, 2025-05-~
$ koen                <fct> Kvinde, Kvinde, Mand, Kvinde, Kvinde, Mand, M~
$ order_trackertag     <chr> "utm_campaign=Marts25&utm_content=b&utm_mediu~
$ permission_given_order <fct> 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
$ permission_given_today <fct> 0, 1, 0, 0, 0, 0, 1, 0, 1, 1, 0, 0, 1, 0, 0, ~
$ previous_subscriptions <int> 5, 15, 0, 0, 2, 4, 9, 0, 12, 5, 0, 0, 21, 1, ~
$ previous_campaigns    <int> 4, 13, 0, 0, 2, 3, 6, 0, 2, 4, 0, 0, 9, 1, 1, ~
$ previous_trials       <int> 2, 1, 0, 0, 0, 0, 4, 0, 6, 1, 0, 0, 10, 0, 0, ~
$ first_campaign_day    <date> 2025-03-05, 2025-03-05, 2025-03-18, 2025-03-~
$ last_campaign_day     <date> 2025-05-04, 2025-05-04, 2025-05-17, 2025-05-~
$ newsletters_before_order <int> 1, 0, 0, 0, 0, 0, 5, 0, 3, 1, 0, 0, 0, 0, 0, ~
$ newsletters_after_order <int> 1, 0, 0, 0, 0, 0, 4, 0, 3, 1, 0, 0, 0, 0, 0, ~
$ type                 <fct> Andet, Andet, Andet, Produkt, Tid, Andet, Tid~
$ reason               <fct> kunden ønsker ikke at oplyse, Ingen årsag - i~
$ expiration_date      <date> 2025-06-04, 2025-05-04, 2025-05-17, 2025-06-~
$ churn                <fct> 0, 1, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 1, 0, ~
$ continued_subscription <dbl> 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 1, 0, 0, 1, ~
$ early_churn           <dbl> 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, ~
$ age                  <dbl> 76.25753, 69.70685, 39.67397, 45.61096, 74.21~
$ kundetid_gruppe     <fct> 6+ måneder, 6+ måneder, 0-7 dage, 0-7 dage, 6~

```

```

# Overblik over churn
model_data %>%

```

```
count(churn) %>%
mutate(prop = n / sum(n))
```

```
# A tibble: 2 x 3
  churn      n prop
  <fct> <int> <dbl>
1 0       731 0.581
2 1       528 0.419
```

```
summary(model_data$account_active_days)
```

```
Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
 1.0   127.5   369.0   473.6   618.5   2847.0
```

```
summary(subscription$account_active_days)
```

```
Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
 1.0   126.5   369.0   473.8   623.5   2847.0
```

```
model_data %>%
count(account_active_days, sort = TRUE) %>%
head(20)
```

```
# A tibble: 20 x 2
  account_active_days      n
      <int> <int>
1           61    163
2           92     48
3          122     30
4          153     23
5          275     20
6          184     17
7          245     16
8          214     13
9          306     13
10         383     13
11         371     12
12         372     10
13         389      9
14         375      8
15         376      7
16          68      6
17         337      6
18         370      6
19         373      6
20         380      6
```

```
model_data %>%
group_by(churn) %>%
summarise(
  mean = mean(account_active_days),
  median = median(account_active_days),
  sd = sd(account_active_days)
)
```

```

# A tibble: 2 x 4
  churn mean median    sd
<fct> <dbl> <dbl> <dbl>
1 0      526.    377 451.
2 1      400.    235 429.

# Outlier tjek - dem har vi fjernet fra model_data
outliers <- merged_data %>%
  mutate(
    order_date = as.Date(ymd_hms(order_date)),
    birthdate = as.Date(parse_date_time(birthdate, orders = c("dmy", "ymd", "mdy"))),
    usr_created = dmy(usr_created),

    age = as.numeric(Sys.Date() - birthdate) / 365,
    kundetid_dage = as.numeric(order_date - usr_created)
  ) %>%
  filter(
    age < 15 | age > 105 |
    kundetid_dage < 0 |
    is.na(age) |
    is.na(kundetid_dage)
  ) %>%
  select(pseudo_id, birthdate, age, usr_created, order_date, kundetid_dage)

# Kig outliers
glimpse(outliers)

```

```

Rows: 19
Columns: 6
$ pseudo_id    <chr> "04a08cd300c7b1c80803fac3de32df92", "10dccc12a352afbf406~
$ birthdate    <date> 2026-01-01, 1992-05-02, 2025-03-12, 1940-10-10, 1883-05~
$ age          <dbl> 0.3287671, 34.0191781, 1.1369863, 85.6136986, 143.095890~
$ usr_created  <date> 2013-04-22, NA, 2023-05-27, NA, 2025-03-08, 2025-03-17,~
$ order_date   <date> 2025-03-12, 2025-03-04, 2025-03-21, 2025-03-15, 2025-03~
$ kundetid_dage <dbl> 4342, NA, 664, NA, 0, -2, 3084, 3559, -32, -218, NA, -1,~

```

```

# Gem data
saveRDS(model_data, "data/model_data.rds")

```

6.2 Klyngeanalyse

```

pacman::p_load(tidyverse, DataExplorer, ggpubr)

# Load data
behavior <- read.csv("data/behavior.csv")
model_data <- readRDS("data/model_data.rds")

# Vi formaterer data så vi kan join det med vores model_data
customer_behavior <- behavior %>%
  group_by(pseudo_id) %>%
  summarise(
    n_visits = n(),

```

```

    avg_scroll = mean(scroll_depth, na.rm = TRUE),
    share_mobile = mean(dvce_type == "Mobile"),
    share_desktop = mean(dvce_type == "Computer"),
    share_restricted = mean(page_restricted == "yes"),
    share_search = mean(refr_medium == "search"),
    share_internal = mean(refr_medium == "internal"),
    n_unique_pages = n_distinct(webpage_id)
  )

saveRDS(customer_behavior, "data/coolbehavior.rds")

# Vi vælger værdier fra model_data som skal bruges til klyngeanalysen til join
subscription_features <- model_data %>%
  select(
    pseudo_id,
    account_active_days,
    previous_subscriptions,
    previous_campaigns,
    previous_trials,
    newsletters_before_order,
    newsletters_after_order,
    churn
  )

# Vi joiner tabellerne sammen til klyngeanalysen
customer_df <- subscription_features %>%
  left_join(customer_behavior, by = "pseudo_id")

# Indsæt 0 i stedet for NA for de 70 "Spørgelses-brugere"
# Siden der er nogle brugere som ikke har behavior værdier,
# sætter vi dem til 0 for at beholde rækkerne
customer_df <- customer_df %>%
  mutate(across(
    c(n_visits, n_unique_pages, avg_scroll, starts_with("share-")),
    ~replace_na(.x, 0)
  ))

# check om vi har NA værdier
colSums(is.na(customer_df))

```

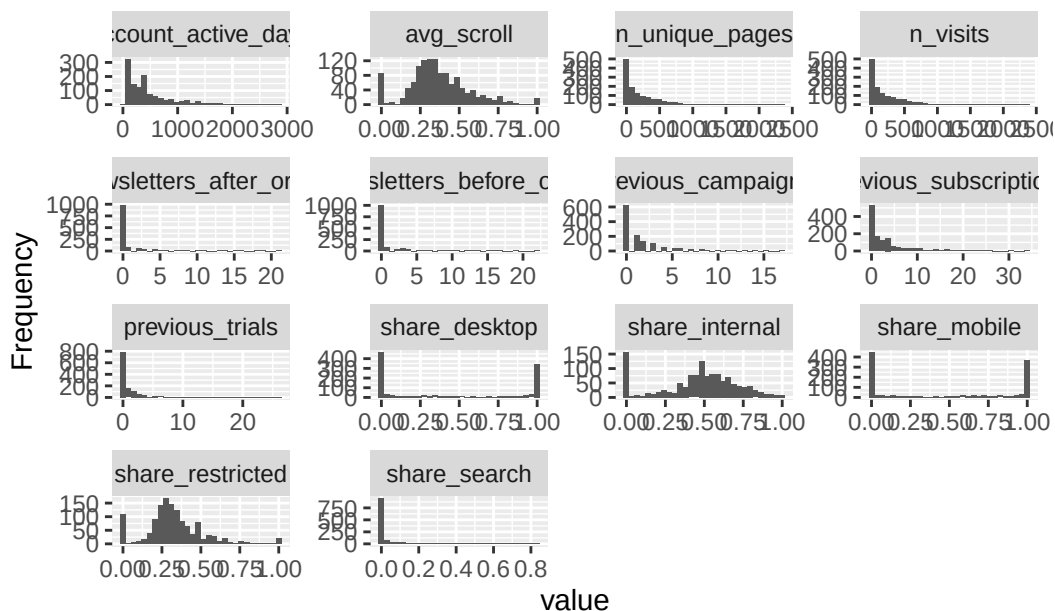
pseudo_id	account_active_days	previous_subscriptions
0	0	0
previous_campaigns	previous_trials	newsletters_before_order
0	0	0
newsletters_after_order	churn	n_visits
0	0	0
avg_scroll	share_mobile	share_desktop
0	0	0
share_restricted	share_search	share_internal
0	0	0
n_unique_pages		
0		

```
# Nu fjerner vi pseudo_ID
cluster_input <- customer_df %>%
  select(-pseudo_id, -churn)

# Tjek data som ryger ind i modellen
glimpse(cluster_input)
```

```
Rows: 1,259
Columns: 14
$ account_active_days      <int> 387, 1330, 61, 92, 581, 1424, 1501, 184, 1256~
$ previous_subscriptions   <int> 5, 15, 0, 0, 2, 4, 9, 0, 12, 5, 0, 0, 21, 1, ~
$ previous_campaigns       <int> 4, 13, 0, 0, 2, 3, 6, 0, 2, 4, 0, 0, 9, 1, 1,~
$ previous_trials          <int> 2, 1, 0, 0, 0, 0, 4, 0, 6, 1, 0, 0, 10, 0, 0,~
$ newsletters_before_order <int> 1, 0, 0, 0, 0, 0, 5, 0, 3, 1, 0, 0, 0, 0, 0,~
$ newsletters_after_order  <int> 1, 0, 0, 0, 0, 0, 4, 0, 3, 1, 0, 0, 0, 0, 0,~
$ n_visits                 <int> 517, 603, 15, 8, 46, 253, 201, 15, 80, 772, 2~
$ avg_scroll               <dbl> 0.3471954, 0.4179104, 0.3666667, 0.6250000, 0~
$ share_mobile             <dbl> 0.47969052, 0.83084577, 1.00000000, 1.0000000~
$ share_desktop            <dbl> 0.5203095, 0.00000000, 0.00000000, 0.0000000, 1~
$ share_restricted         <dbl> 0.2978723, 0.3200663, 0.3333333, 0.6250000, 0~
$ share_search             <dbl> 0.015473888, 0.439469320, 0.000000000, 0.0000~
$ share_internal           <dbl> 0.52417795, 0.53565506, 0.46666667, 0.0000000~
$ n_unique_pages           <int> 516, 603, 15, 8, 46, 253, 201, 15, 80, 771, 2~
```

```
# Tjek eventuelt histogram over tallene
plot_histogram(cluster_input)
```



```
# Kør scale til klyngeanalysen
customer_scaled <- scale(cluster_input)

# Principal component analyse
pca <- prcomp(customer_scaled, scale = TRUE)
```

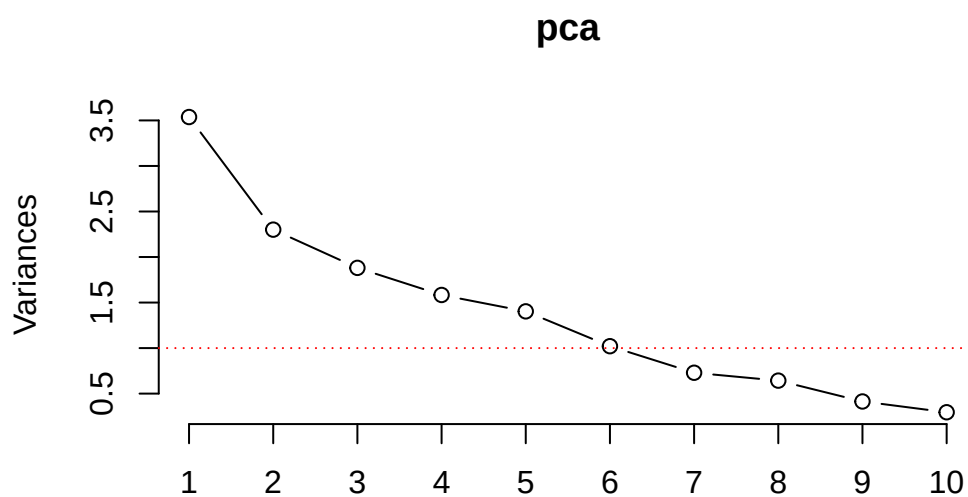
```
summary(pca)
```

Importance of components:

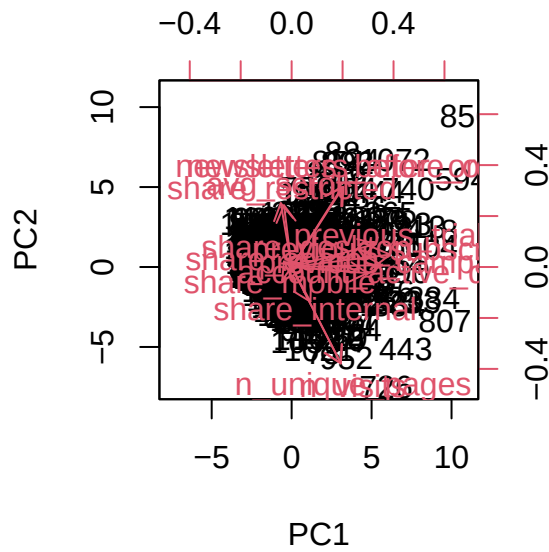
	PC1	PC2	PC3	PC4	PC5	PC6	PC7
Standard deviation	1.8808	1.5168	1.3714	1.2582	1.1846	1.01035	0.85395
Proportion of Variance	0.2527	0.1643	0.1343	0.1131	0.1002	0.07292	0.05209
Cumulative Proportion	0.2527	0.4170	0.5514	0.6644	0.7647	0.83758	0.88967

	PC8	PC9	PC10	PC11	PC12	PC13	PC14
Standard deviation	0.80168	0.64323	0.54330	0.34587	0.22191	0.15005	0.04036
Proportion of Variance	0.04591	0.02955	0.02108	0.00854	0.00352	0.00161	0.00012
Cumulative Proportion	0.93558	0.96513	0.98621	0.99476	0.99828	0.99988	1.00000

```
screeplot(pca, type = "line")  
abline(h = 1, col = "red", lty = 3)
```



```
biplot(pca, scale = 0)
```

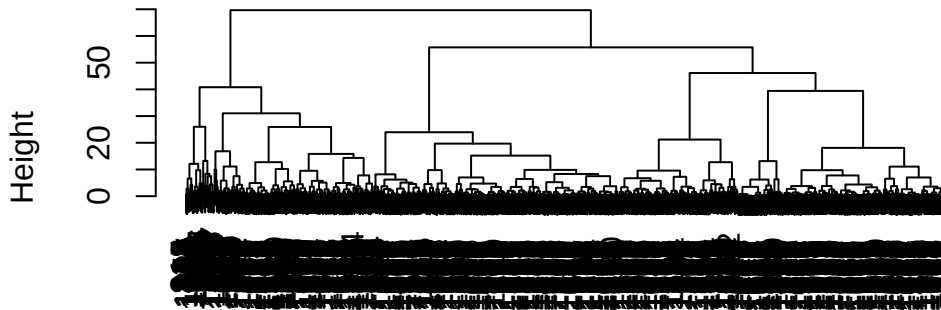


```
# Hierarchical clustering

# hc_complete <- hclust(dist(pca$x[,1:4]), method = "complete")
# plot(hc_complete)
#
# customer_df <- customer_df %>%
#   filter(complete.cases(select(., -pseudo_id, -churn)))
#
# customer_df$cluster_hc <- cutree(hc_complete, 5)
#
# table(customer_df$cluster_hc)

hc_ward <- hclust(dist(pca$x[,1:4]), method = "ward.D2")
plot(hc_ward)
```

Cluster Dendrogram



```
dist(pca$x[, 1:4])
hclust (*, "ward.D2")
```

```
customer_df <- customer_df %>%
  filter(complete.cases(select(., -pseudo_id, -churn)))

customer_df$cluster_hc <- cutree(hc_ward, 5)

table(customer_df$cluster_hc)
```

```
  1    2    3    4    5
265 342 407 198  47
```

```
# K-means clustering
set.seed(8)

km <- kmeans(pca$x[,1:4], centers = 5, nstart = 20)

customer_df$cluster_km <- km$cluster

table(customer_df$cluster_km)
```

```
  1    2    3    4    5
239 418  91 365 146
```

```
# compare methods
table(customer_df$cluster_km, customer_df$cluster_hc)
```

```
      1    2    3    4    5
1  59    0   27  153    0
2  48    5  361    1    3
3   7   75    9    0    0
```

```
4 49 262 9 44 1
5 102 0 1 0 43
```

```
# Klyngeprofiling
customer_df$cluster_km <- as.factor(customer_df$cluster_km)

cluster_profile <- customer_df %>%
  group_by(cluster_km) %>%
  summarise(across(where(is.numeric), mean, na.rm = TRUE))
```

Warning: There was 1 warning in `summarise()`.

- i In argument: `across(where(is.numeric), mean, na.rm = TRUE)`.
- i In group 1: `cluster\_km = 1`.

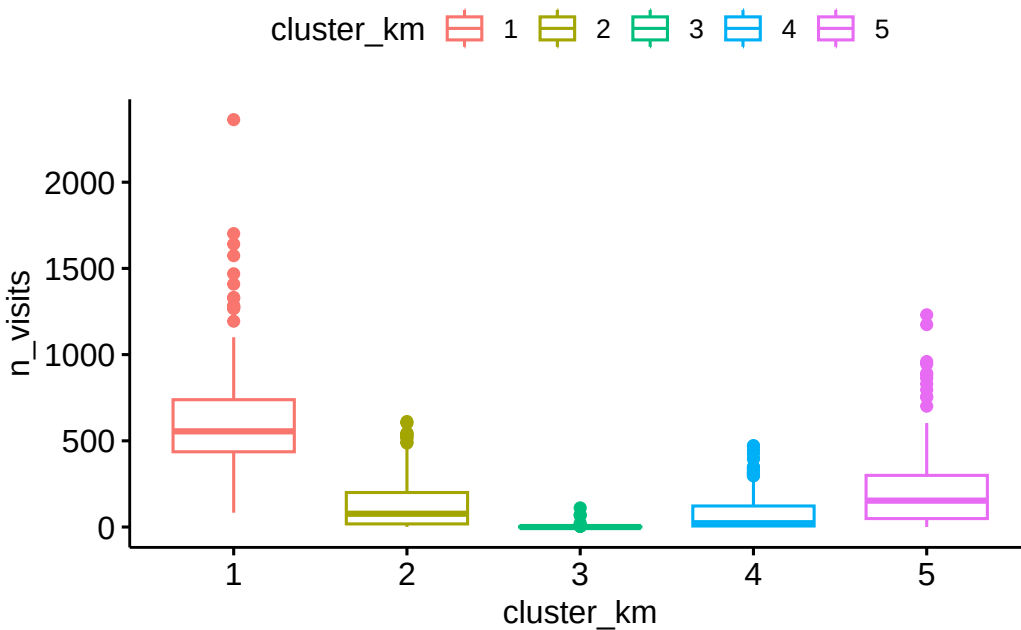
Caused by warning:

- ! The `...` argument of `across()` is deprecated as of dplyr 1.1.0. Supply arguments directly to `.fns` through an anonymous function instead.

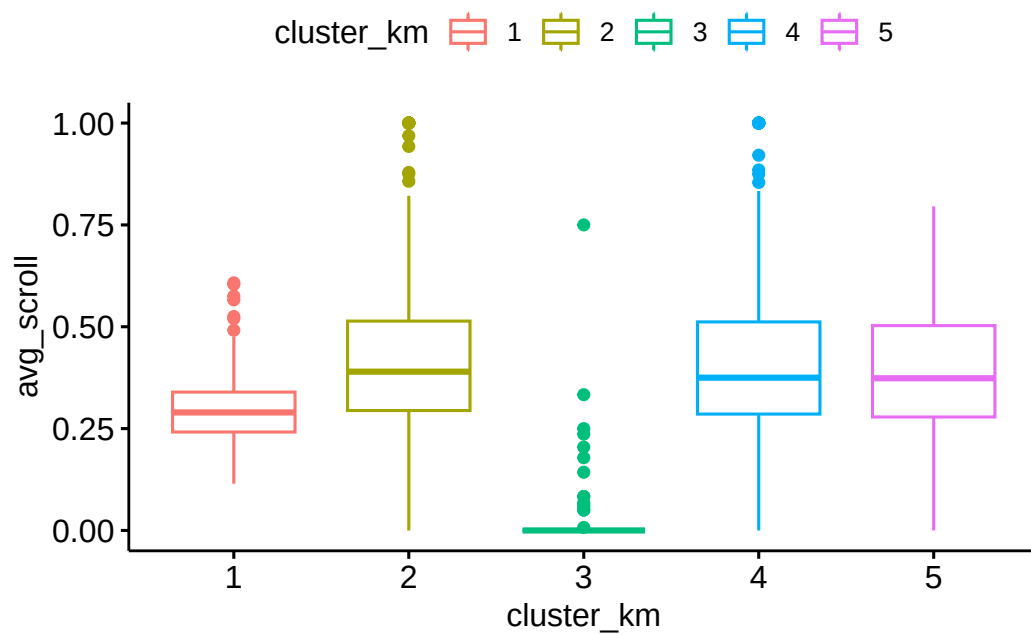
```
# Previously
across(a:b, mean, na.rm = TRUE)

# Now
across(a:b, \(x) mean(x, na.rm = TRUE))
```

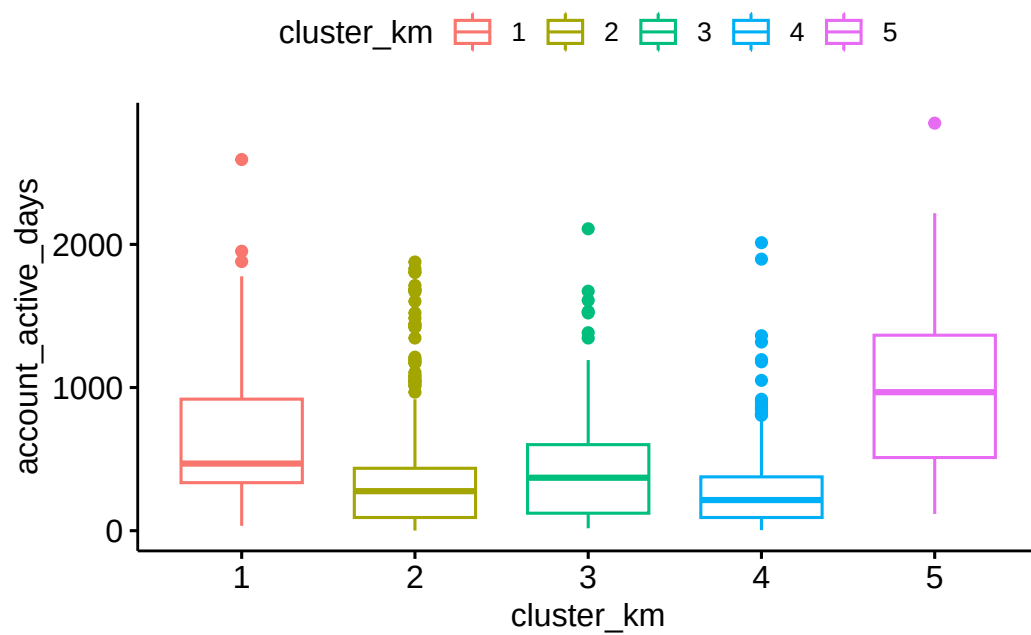
```
# Visualisering
ggboxplot(customer_df, x = "cluster_km", y = "n_visits",
  color = "cluster_km")
```



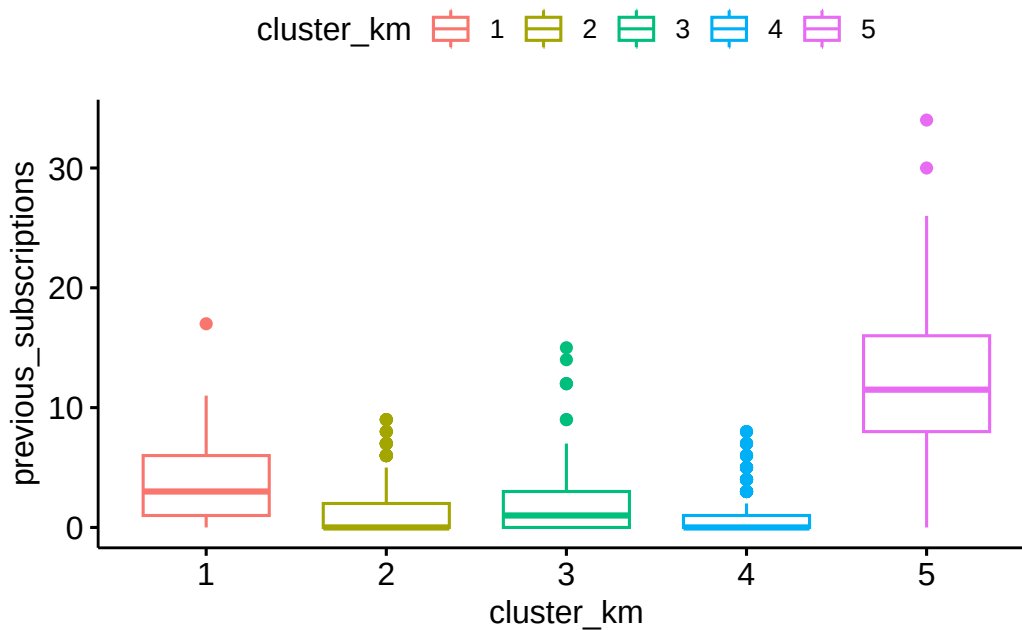
```
ggboxplot(customer_df, x = "cluster_km", y = "avg_scroll",
  color = "cluster_km")
```



```
ggboxplot(customer_df, x = "cluster_km", y = "account_active_days",
           color = "cluster_km")
```



```
ggboxplot(customer_df, x = "cluster_km", y = "previous_subscriptions",
           color = "cluster_km")
```



```
# ANOVA tests
```

```
summary(aov(n_visits ~ cluster_km, data = customer_df))
```

```

      Df   Sum Sq Mean Sq F value Pr(>F)
cluster_km    4 52749585 13187396   393 <2e-16 ***
Residuals  1254 42084141    33560
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
summary(aov(avg_scroll ~ cluster_km, data = customer_df))
```

```

      Df Sum Sq Mean Sq F value Pr(>F)
cluster_km    4  14.18   3.546  129.6 <2e-16 ***
Residuals  1254  34.31   0.027
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
summary(aov(account_active_days ~ cluster_km, data = customer_df))
```

```

      Df   Sum Sq Mean Sq F value Pr(>F)
cluster_km    4 63885784 15971446  107.5 <2e-16 ***
Residuals  1254 186284206   148552
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
summary(aov(previous_subscriptions ~ cluster_km, data = customer_df))
```

```

      Df Sum Sq Mean Sq F value Pr(>F)
cluster_km    4  15416   3854  387.5 <2e-16 ***
Residuals  1254  12473    10
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
summary(aov(previous_campaigns ~ cluster_km, data = customer_df))
```

```

      Df Sum Sq Mean Sq F value Pr(>F)
cluster_km      4   3324    831.0   251.1 <2e-16 ***
Residuals    1254   4151      3.3
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

```
summary(aov(previous_trials ~ cluster_km, data = customer_df))
```

```

      Df Sum Sq Mean Sq F value Pr(>F)
cluster_km      4   4161   1040   258.7 <2e-16 ***
Residuals    1254   5042      4
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

```
summary(aov(newsletters_before_order ~ cluster_km, data = customer_df))
```

```

      Df Sum Sq Mean Sq F value Pr(>F)
cluster_km      4   1551   387.8   119 <2e-16 ***
Residuals    1254   4088      3.3
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

```
summary(aov(newsletters_after_order ~ cluster_km, data = customer_df))
```

```

      Df Sum Sq Mean Sq F value Pr(>F)
cluster_km      4   1607   401.9  109.7 <2e-16 ***
Residuals    1254   4595      3.7
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

```

# Churn analyse
churn_by_cluster <- customer_df %>%
  group_by(cluster_km) %>%
  summarise(
    churn_rate = mean(churn == 1, na.rm = TRUE),
    n = n()
  )

print(churn_by_cluster)

```

```

# A tibble: 5 x 3
  cluster_km churn_rate      n
  <fct>      <dbl> <int>
1 1          0.469   239
2 2          0.419   418
3 3          0.385    91
4 4          0.362   365
5 5          0.507   146

```

```
# Print summary af klynger
cluster_summary_long <- customer_df %>%
  group_by(cluster_km) %>%
  summarise(across(where(is.numeric), \(x) mean(x, na.rm = TRUE))) %>%
  pivot_longer(cols = -cluster_km, names_to = "variable", values_to = "mean_value") %>%
  pivot_wider(names_from = cluster_km, names_prefix = "Cluster_", values_from = mean_value)

print(cluster_summary_long, n = 50)
```

```
# A tibble: 15 x 6
  variable          Cluster_1 Cluster_2 Cluster_3 Cluster_4 Cluster_5
  <chr>             <dbl>    <dbl>    <dbl>    <dbl>    <dbl>
1 account_active_days 640.    369.    469.    280.    990.
2 previous_subscriptions 3.65    1.38    2.32    0.989   12.3
3 previous_campaigns    2.05    0.821   1.27    0.573    5.84
4 previous_trials       1.28    0.486   0.901   0.384    6.24
5 newsletters_before_order 0.331   0.292   0.330   0.395    3.80
6 newsletters_after_order 0.377   0.340   0.363   0.471    3.92
7 n_visits            617.    126.    4.02    78.2    228.
8 avg_scroll           0.299   0.423   0.0281  0.423    0.400
9 share_mobile          0.686   0.0644  0.168   0.904    0.414
10 share_desktop         0.271   0.908   0.0963  0.0674   0.544
11 share_restricted      0.277   0.348   0.0169  0.394    0.357
12 share_search          0.0490  0.0683  0.00139 0.0407   0.0353
13 share_internal        0.568   0.538   0.0545  0.423    0.507
14 n_unique_pages        614.    126.    4.02    77.7    228.
15 cluster_hc            3.15    2.78    2.02    2.14    2.19
```

```
# definering af 5 labels baseret på profilerne
customer_df <- customer_df %>%
  mutate(cluster_label = case_when(
    cluster_km == 1 ~ "Power-brugeren (Mest mobil)",
    cluster_km == 2 ~ "Desktop-traditionalisten",
    cluster_km == 3 ~ "Veteranen (Høj historik)",
    cluster_km == 4 ~ "Mobil-nykommeren",
    cluster_km == 5 ~ "Spørgelses-brugeren (Lav aktivitet)",
    TRUE ~ "Andet"
  ))
```

```
# Tjek den nye fordeling
table(customer_df$cluster_label, customer_df$churn)
```

	0	1
Desktop-traditionalisten	243	175
Mobil-nykommeren	233	132
Power-brugeren (Mest mobil)	127	112
Spørgelses-brugeren (Lav aktivitet)	72	74
Veteranen (Høj historik)	56	35

```
# Gem mapping til din Tidymodels model
cluster_mapping <- customer_df %>%
  select(pseudo_id, cluster_label)
```

```
saveRDS(cluster_mapping, "data/cluster_mapping.rds")
```

```
# Tjek fordelingen
table(customer_df$cluster_label)
```

	Desktop-traditionalisten	Mobil-nykommeren
	418	365
Power-brugeren (Mest mobil)	Spørgelses-brugeren (Lav aktivitet)	
	239	146
Veteranen (Høj historik)		
	91	

6.3 Churn model

```
pacman::p_load(
  tidyverse, tidymodels, themis, ranger, glmnet,
  kernlab, kknn, xgboost, naivebayes, rpart, future, vip
)

# Indlæs data
model_data <- readRDS("data/model_data.rds")
cluster_mapping <- readRDS("data/cluster_mapping.rds")
coolbehavior <- readRDS("data/coolbehavior.rds")

### dashboard data til power BI
final_dashboard_data <- model_data %>%
  left_join(cluster_mapping, by = "pseudo_id")

final_dashboard_data$page <- round(final_dashboard_data$page)

merged_dashboard_data <- final_dashboard_data %>%
  left_join(coolbehavior, by = "pseudo_id")

# Indsæt 0 i stedet for NA for de 70 "Spørgelses-brugere"
merged_dashboard_data <- merged_dashboard_data %>%
  mutate(across(
    c(n_visits, n_unique_pages, avg_scroll, starts_with("share_")),
    ~replace_na(.x, 0)
  ))

write_csv(merged_dashboard_data, "data/dashboard_data.csv")

# Join cluster labels and clean initial data
model_data_clean <- model_data %>%
  left_join(cluster_mapping, by = "pseudo_id") %>%
  mutate(cluster_label = as.factor(cluster_label)) %>%
  select(
    -pseudo_id,
    -subscription_cancel_date,
    -type,
    -reason,
    -expiration_date,
```

```

    -order_trackertag,
    -first_campaign_day,
    -last_campaign_day,
    -continued_subscription
  ) %>%
  mutate(churn = factor(churn, levels = c("1", "0")))

# Check that cluster_label is present
glimpse(model_data_clean)

```

```

Rows: 1,259
Columns: 14
$ account_active_days      <int> 387, 1330, 61, 92, 581, 1424, 1501, 184, 1256~
$ koen                     <fct> Kvinde, Kvinde, Mand, Kvinde, Kvinde, Mand, M~
$ permission_given_order   <fct> 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
$ permission_given_today   <fct> 0, 1, 0, 0, 0, 0, 1, 0, 1, 1, 0, 0, 1, 0, ~
$ previous_subscriptions    <int> 5, 15, 0, 0, 2, 4, 9, 0, 12, 5, 0, 0, 21, 1, ~
$ previous_campaigns        <int> 4, 13, 0, 0, 2, 3, 6, 0, 2, 4, 0, 0, 9, 1, 1,~
$ previous_trials           <int> 2, 1, 0, 0, 0, 0, 4, 0, 6, 1, 0, 0, 10, 0, 0,~
$ newsletters_before_order <int> 1, 0, 0, 0, 0, 0, 5, 0, 3, 1, 0, 0, 0, 0, 0, ~
$ newsletters_after_order  <int> 1, 0, 0, 0, 0, 0, 4, 0, 3, 1, 0, 0, 0, 0, 0, ~
$ churn                    <fct> 0, 1, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 1, 0, ~
$ early_churn               <dbl> 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, ~
$ age                      <dbl> 76.25753, 69.70685, 39.67397, 45.61096, 74.21~
$ kundetid_gruppe          <fct> 6+ måneder, 6+ måneder, 0-7 dage, 0-7 dage, 6~
$ cluster_label            <fct> Power-brugeren (Mest mobil), Spørgelses-bruger~

```

```

# Churn fordeling
model_data_clean %>%
  count(churn) %>%
  mutate(prop = n / sum(n))

```

```

# A tibble: 2 x 3
  churn     n prop
  <fct> <int> <dbl>
1 1       528 0.419
2 0       731 0.581

```

```

# Gem data
saveRDS(model_data_clean, "data/model_data_clean.rds")

# Split
set.seed(8)
split <- initial_split(model_data_clean, prop = 0.8, strata = churn)
train_data <- training(split)
test_data <- testing(split)

# CV
folds <- vfold_cv(train_data, v = 5, strata = churn)

# Recipe
rec <- recipe(churn ~ ., data = train_data) %>%
  step_rm(early_churn) %>%
  step_novel(all_nominal_predictors()) %>%

```

```

step_dummy(all_nominal_predictors(), one_hot = TRUE) %>%
step_zv(all_predictors()) %>%
step_normalize(all_numeric_predictors()) %>%
step_smote(churn)
# step_upsample(churn)
# step_downsample(churn)

# Models
decision_tree_spec <- decision_tree(
  tree_depth = tune(),
  min_n = tune(),
  cost_complexity = tune()
) %>%
  set_engine("rpart") %>%
  set_mode("classification")

log_reg_spec <- logistic_reg(
  penalty = tune(),
  mixture = tune()
) %>%
  set_engine("glmnet") %>%
  set_mode("classification")

rand_forest_spec <- rand_forest(
  mtry = tune(),
  min_n = tune(),
  trees = 500
) %>%
  set_engine("ranger", importance = "permutation") %>%
  set_mode("classification")

svm_rbf_spec <- svm_rbf(
  cost = tune(),
  rbf_sigma = tune()
) %>%
  set_engine("kernlab") %>%
  set_mode("classification")

xgb_spec <- boost_tree(
  trees = tune(),
  tree_depth = tune(),
  learn_rate = tune(),
  mtry = tune()
) %>%
  set_engine("xgboost") %>%
  set_mode("classification")

# Workflow
workflow_set_obj <- workflow_set(
  preproc = list(rec = rec),
  models = list(
    decision_tree = decision_tree_spec,
    logistic_reg = log_reg_spec,
    rand_forest = rand_forest_spec,
    svm_rbf = svm_rbf_spec,

```

```

    xgboost      = xgb_spec
  )
)

metrics <- metric_set(roc_auc, accuracy, f_meas, sens, spec)

# Train
plan(multisession)

set.seed(8)

results <- suppressMessages(suppressWarnings(
  workflow_set_obj %>%
    workflow_map(
      "tune_grid",
      resamples = folds,
      grid = 10,
      metrics = metrics,
      verbose = FALSE,
      control = control_grid(
        save_pred = TRUE,
        save_workflow = TRUE
      )
    )
  ))

```

maximum number of iterations reached 1.360417e-05 -8.598677e-13maximum number of iterations reached 0.0

```

plan(sequential)

# Best model
best_model_id <- results %>%
  rank_results(select_best = TRUE) %>%
  filter(.metric == "f_meas") %>%
  slice_max(mean, n = 1) %>%
  pull(wflow_id)

best_tuned <- results %>%
  extract_workflow_set_result(best_model_id) %>%
  select_best(metric = "f_meas")

final_wf <- results %>%
  extract_workflow(best_model_id) %>%
  finalize_workflow(best_tuned)

# Final evaluation
final_fit <- final_wf %>%
  last_fit(split, metrics = metrics)

collect_metrics(final_fit)

```

```

# A tibble: 5 x 4
  .metric .estimator .estimate .config

```

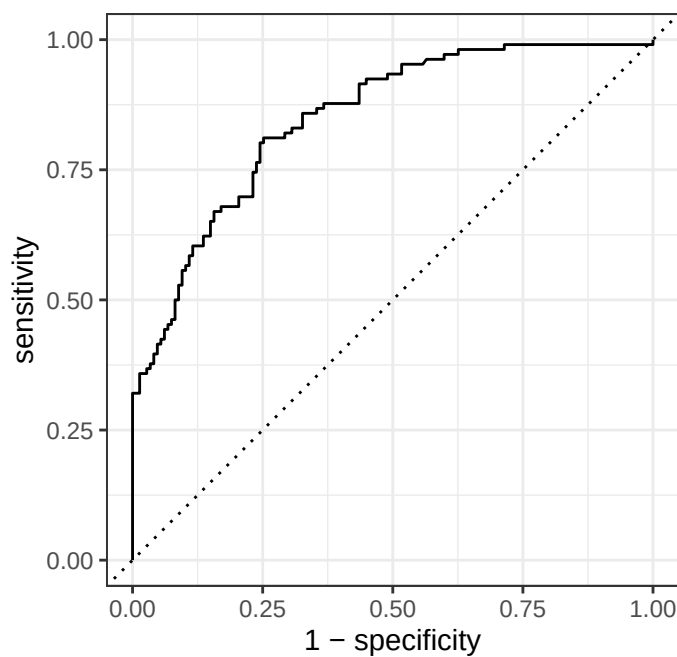
	<chr>	<chr>	<dbl>	<chr>
1	accuracy	binary	0.771	pre0_mod0_post0
2	f_meas	binary	0.743	pre0_mod0_post0
3	sens	binary	0.792	pre0_mod0_post0
4	spec	binary	0.755	pre0_mod0_post0
5	roc_auc	binary	0.849	pre0_mod0_post0

```
test_preds <- collect_predictions(final_fit)

test_preds %>%
  conf_mat(truth = churn, estimate = .pred_class)
```

	Truth	
Prediction	1	0
1	84	36
0	22	111

```
test_preds %>%
  roc_curve(truth = churn, .pred_1) %>%
  autoplot()
```



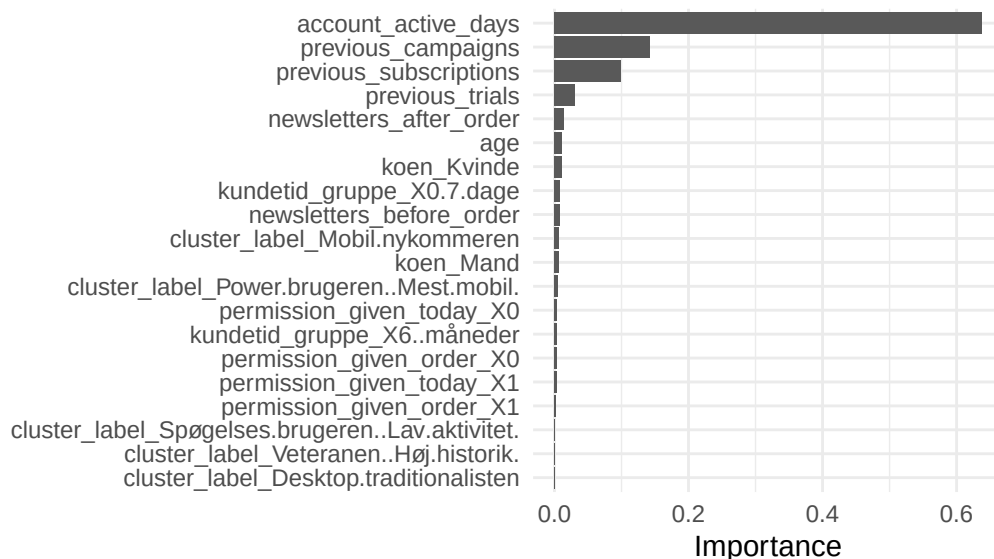
```
best_model_id
```

```
[1] "rec_xgboost"
```

```
final_fit %>%
  extract_fit_parsnip() %>%
  vip(num_features = 20) +
  theme_minimal() +
  labs(title = "What Drives Churn?",
       subtitle = "Variables ranked by Importance (Random Forest)")
```

What Drives Churn?

Variables ranked by Importance (Random Forest)



```
#####
#  MODEL 2: EARLY CHURN (TASK 3)
#####

# Only customers who continued
early_data <- model_data_clean %>%
  filter(churn == "0") %>%
  mutate(early_churn = factor(early_churn, levels = c("0", "1")))

# Split
set.seed(8)
split_early <- initial_split(early_data, prop = 0.8, strata = early_churn)
train_early <- training(split_early)
test_early <- testing(split_early)

# CV
folds_early <- vfold_cv(train_early, v = 5, strata = early_churn)

# Recipe
rec_early <- recipe(early_churn ~ ., data = train_early) %>%
  step_rm(churn) %>% # remove parent target
  step_novel(all_nominal_predictors()) %>%
  step_dummy(all_nominal_predictors(), one_hot = TRUE) %>%
  step_zv(all_predictors()) %>%
  step_normalize(all_numeric_predictors())
# step_downsample(early_churn)

# Workflow (reuse same models!)
workflow_set_early <- workflow_set(
  preproc = list(rec = rec_early),
  models = list(
    decision_tree = decision_tree_spec,
    logistic_reg = log_reg_spec,
```

```

    rand_forest = rand_forest_spec,
    svm_rbf     = svm_rbf_spec,
    xgboost     = xgb_spec
  )
)

# Train
plan(multisession)

set.seed(8)
results_early <- workflow_set_early %>%
  workflow_map(
    "tune_grid",
    resamples = folds_early,
    grid = 10,
    metrics = metrics
  )

```

Warning: package 'future' was built under R version 4.5.3

Warning: package 'tune' was built under R version 4.5.2

Warning: package 'future' was built under R version 4.5.3

Warning: package 'tune' was built under R version 4.5.2

Warning: package 'future' was built under R version 4.5.3

Warning: package 'tune' was built under R version 4.5.2

Warning: package 'future' was built under R version 4.5.3

Warning: package 'tune' was built under R version 4.5.2

Warning: package 'future' was built under R version 4.5.3

Warning: package 'tune' was built under R version 4.5.2

i Creating pre-processing data to finalize 1 unknown parameter: "mtry"

maximum number of iterations reached 0.003258396 0.00318488maximum number of iterations reached 0.00024

i Creating pre-processing data to finalize 1 unknown parameter: "mtry"

```

plan(sequential)

# Best model
best_model_id_early <- results_early %>%
  rank_results(select_best = TRUE) %>%
  filter(.metric == "f_meas") %>%
  slice_max(mean, n = 1) %>%

```

```

pull(wflow_id)

best_tuned_early <- results_early %>%
  extract_workflow_set_result(best_model_id_early) %>%
  select_best(metric = "f_meas")

final_wf_early <- results_early %>%
  extract_workflow(best_model_id_early) %>%
  finalize_workflow(best_tuned_early)

# Final evaluation
final_fit_early <- final_wf_early %>%
  last_fit(split_early, metrics = metrics)

collect_metrics(final_fit_early)

```

```

# A tibble: 5 x 4
  .metric .estimator .estimate .config
  <chr>    <chr>         <dbl> <chr>
1 accuracy binary         0.837 pre0_mod0_post0
2 f_meas  binary         0.888 pre0_mod0_post0
3 sens    binary         0.95  pre0_mod0_post0
4 spec    binary         0.596 pre0_mod0_post0
5 roc_auc binary         0.869 pre0_mod0_post0

```

```

test_preds_early <- collect_predictions(final_fit_early)

test_preds_early %>%
  conf_mat(truth = early_churn, estimate = .pred_class)

```

```

      Truth
Prediction 0 1
      0 95 19
      1  5 28

```

```
best_model_id_early
```

```
[1] "rec_rand_forest"
```

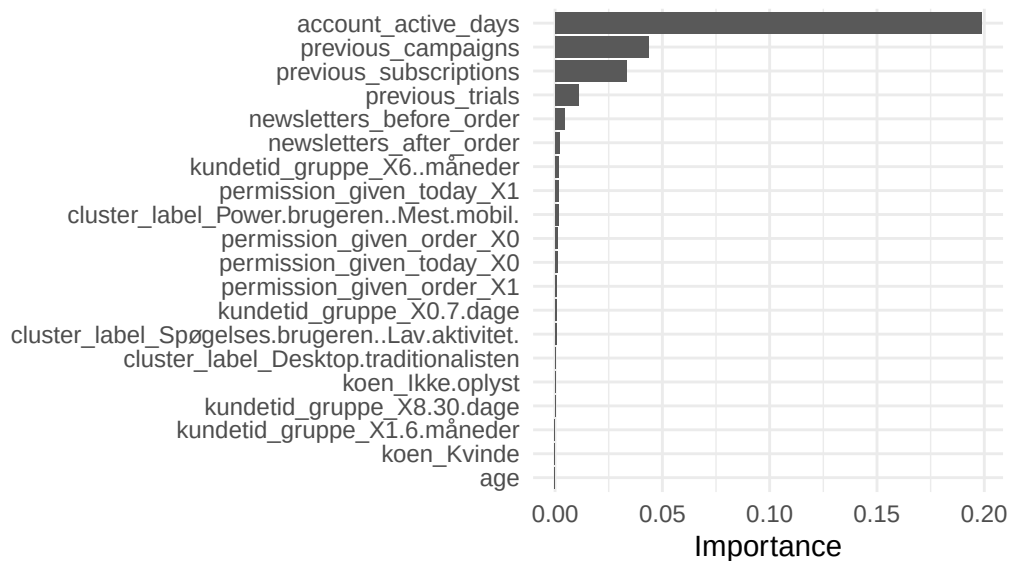
```

final_fit_early %>%
  extract_fit_parsnip() %>%
  vip(num_features = 20) +
  theme_minimal() +
  labs(title = "What Drives Early Churn?",
       subtitle = "Variables ranked by Importance (Decision Tree)")

```

What Drives Early Churn?

Variables ranked by Importance (Deci



```
# Best og worst case definition
# Extract fitted workflows
churn_model <- extract_workflow(final_fit)
early_model <- extract_workflow(final_fit_early)

scenarier <- tibble(
  case = c("Best case", "Average case", "Worst case"),

  # numeric
  account_active_days = c(2000, 450, 5),
  age = c(70, 45, 22),
  previous_subscriptions = c(1, 2, 0),
  previous_campaigns = c(1, 2, 0),
  previous_trials = c(0, 10, 0),
  newsletters_before_order = c(1, 2, 0),
  newsletters_after_order = c(1, 2, 0),

  # Kategoriske værdier
  koen = factor(
    c("Mand", "Mand", "Mand"),
    levels = levels(model_data_clean$koen)
  ),

  kundetid_gruppe = factor(
    c("6+ måneder", "6+ måneder", "0-7 dage"),
    levels = levels(model_data_clean$kundetid_gruppe)
  ),

  cluster_label = factor(
    c(
      "Veteranen (Høj historik)",
      "Desktop-traditionalisten",
      "Mobil-nykommeren"
```

```

    ),
    levels = levels(model_data_clean$cluster_label)
  ),

  permission_given_order = factor(
    c("1", "1", "0"),
    levels = levels(model_data_clean$permission_given_order)
  ),

  permission_given_today = factor(
    c("1", "0", "0"),
    levels = levels(model_data_clean$permission_given_today)
  ),
  churn = factor(c("0", "0", "0"), levels = c("1", "0")),
  early_churn = c(0, 0, 0)
)
# Forudseelser
# Main churn model
p_churn <- predict(
  churn_model,
  scenarier,
  type = "prob"
) %>%
  select(churn_prob = .pred_1)

# Early churn model
p_early <- predict(
  early_model,
  scenarier,
  type = "prob"
) %>%
  select(early_prob = .pred_1)

# Kombiner resultater med bind_cols
scenario_results <- bind_cols(
  scenarier,
  p_churn,
  p_early
) %>%
  mutate(
    total_risk =
      churn_prob +
      (1 - churn_prob) * early_prob
  ) %>%
  mutate(
    across(
      c(churn_prob, early_prob, total_risk),
      ~ round(.x * 100, 1)
    )
  ) %>%
  arrange(total_risk) %>%
  select(
    case,
    total_risk,
    churn_prob,

```

```

    early_prob,
    everything()
)

print(scenario_results, width = Inf)

# A tibble: 3 x 18
  case          total_risk churn_prob early_prob account_active_days  age
<chr>          <dbl>      <dbl>    <dbl>          <dbl> <dbl>
1 Best case      33.7        22.3      14.6            2000   70
2 Average case   66.5        49.7      33.4             450   45
3 Worst case    99.6         93       94.3              5   22
  previous_subscriptions previous_campaigns previous_trials
          <dbl>          <dbl>          <dbl>
1             1             1             0
2             2             2            10
3             0             0             0
  newsletters_before_order newsletters_after_order koen kundetid_gruppe
          <dbl>          <dbl> <fct> <fct>
1             1             1 Mand  6+ måneder
2             2             2 Mand  6+ måneder
3             0             0 Mand  0-7 dage
  cluster_label      permission_given_order permission_given_today churn
<fct>          <fct>          <fct>          <fct>
1 Veteranen (Høj historik) 1             1             0
2 Desktop-traditionalisten 1             0             0
3 Mobil-nykommeren         0             0             0
  early_churn
    <dbl>
1         0
2         0
3         0

```

6.4 Neural network churn model

Åbn <https://colab.research.google.com/>, lav “New Notebook” og importer JP_NN2.ipynb. Derefter skal subscription_v2.csv og cancellation.csv indsættes i Filer under “sample_data”. Derefter kan koden køres.

Vi har her en rensningsblok og derefter fire modeller der i alt fylder 16000 tegn inkl. mellemrum.

Model 01 fungerer som en baseline og er implementeret som et simpelt neuralt netværk ved brug af Keras Sequential API. I koden ses dette bl.a. ved et enkelt skjult lag (Dense(..., activation='relu')) med L2-regularisering samt et outputlag med sigmoid-aktivering. Modellen kompiles med adam og binary_crossentropy. Formålet er at etablere et sammenligningsgrundlag uden brug af avancerede teknikker.

Model 02 udvider baseline-modellen ved at inkludere flere skjulte lag, eksempelvis Dense(64) og Dense(32), samt BatchNormalization(). Dette øger modellens kapacitet til at lære mere komplekse mønstre i data og bidrager samtidig til en mere stabil træningsproces. Modellen repræsenterer derfor en mere avanceret arkitektur end Model 01.

Model 03 fokuserer på håndtering af ubalancerede data ved brug af SMOTE, hvilket i koden ses ved SMOTE().fit_resample(X_train, y_train). Denne metode skaber et mere balanceret træningsdatasæt ved at generere syntetiske observationer af minoritetsklassen. Selve netværksarkitekturen minder om de tidligere modeller, men træningen foregår på et forbedret datagrundlag, hvilket styrker modellens evne til at identificere churn-kunder.

Model 04 anvender SMOTENC (SMOTENC(categorical_features=...)) som oversampling-teknik. Denne metode tager højde for både numeriske og kategoriske variable og skaber dermed mere realistiske syntetiske observa-

tioner sammenlignet med standard SMOTE. Modellen kombinerer derfor en neural netværksarkitektur med en mere avanceret tilgang til håndtering af datasæt-ubalance.

Til sidst vurderes modellerne samlet ved hjælp af ROC-kurver, confusion matrices og centrale performance-mål såsom AUC, recall og accuracy.

Resultaterne viser, at Model 04 (SMOTENC) opnår den bedste samlede performance med den højeste AUC på 0,792 samt den højeste recall på 0,783. Det betyder, at modellen er bedst til at identificere churn-kunder, hvilket ofte er det vigtigste mål i churn-prediktion, da det er mere værdifuldt at fange flest mulige potentielle kunder, der risikerer at forlade virksomheden.

Model 01 (Baseline) præsterer overraskende stærkt med en AUC på 0,779 og den højeste accuracy på 0,694, hvilket indikerer, at den simple model fortsat er konkurrencedygtig. Model 03 (SMOTE) opnår samme AUC på 0,779, men en højere recall på 0,755, hvilket viser, at SMOTE forbedrer modellens evne til at identificere churn-kunder på bekostning af en mindre reduktion i accuracy.

Model 02 (Complex) opnår de svageste resultater med en AUC på 0,759 og en recall på 0,613, hvilket tyder på, at øget modelkompleksitet alene ikke nødvendigvis fører til bedre performance. Tværtimod peger resultaterne på, at håndtering af datasæt-ubalance har større betydning end en mere kompleks netværksarkitektur.

Samlet set fremstår Model 04 (SMOTENC) som den stærkeste model, da den både leverer den bedste samlede diskriminationsevne (AUC) og den højeste recall. Resultaterne understreger dermed, at avancerede metoder til håndtering af ubalancerede data kan have en væsentlig positiv effekt på churn-prediktion.